

МОСКОВСКИЙ АВИАЦИОННЫЙ ИНСТИТУТ
(НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ)

Институт №8 «Компьютерные науки и прикладная математика»

Лабораторная работа №1
по курсу «Программирование Графических Процессоров»

**Освоение программного обеспечения для работы с технологией
CUDA.**
Примитивные операции над векторами.

Выполнил: О.С. Концебалов
Группа: М8О-409Б-22
Преподаватели: А.Ю. Морозов,
Е.Е. Заяц

Москва, 2025

Условие

Цель работы: Ознакомление и установка программного обеспечения для работы с программно-аппаратной архитектурой параллельных вычислений (CUDA). Реализация одной из примитивных операций над векторами.

В качестве вещественного типа данных необходимо использовать тип данных double. Все результаты выводить с относительной точностью 10^{-10} . Ограничение: $n < 2^{25}$

Вариант 8. Реверс вектора

Входные данные. На первой строке задано число n -- размер векторов. На следующей строке записано n вещественных чисел -- элементы вектора.

Выходные данные. Необходимо вывести n чисел -- результат реверса исходного вектора.

Пример:

Входной файл	Выходной файл
3 1 2 3	3.0000000000e+00 2.0000000000e+00 1.0000000000e+00

Программное и аппаратное обеспечение

GPU

Compute capability	8.6
Name	NVIDIA GeForce RTX 3050 Ti Laptop GPU
Total Global Memory	4294443008
Shared memory per block	49152
Registers per block	65536
Warp size	32
Max threads per block	(1024, 1024, 64)
Max block	(2147483647, 65535, 65535)
Total constant memory	65536
Multiprocessors count	20

Метод решения

В программе в динамической памяти был выделен массив длины n . Далее было проведено заполнение этого массива входными данными, после чего на видеокарте была выделена память для копирования уже созданного массива данных. Далее после копирования массива в память видеокарты было вызвано ядро, которое в свою очередь произвело реверс вектора. Далее данные из видеокарты копировались обратно в память процессора, после чего выводился ответ в указанном формате.

Описание программы

Программа состоит из функции «main» и ядра. Ядро принимает указатель на массив данных, который нужно реверснуть, и его размер. Также в ядре вычисляется абсолютный индекс потока, благодаря которому каждому потоку назначаются элементы массива для обмена значениями. Этот процесс зациклен, то есть на каждый поток назначаются индексы для обмена до тех пор, пока номер назначенного индекса не выйдет за пределы размерности массива, то есть n . Таким образом каждый поток при достаточно больших n может производить обмен значениями нескольких элементов вектора.

Результаты

$n = 1\ 000$

```
|grids: 1|blocks: 32|time: 0.072644 ms|
|grids: 1|blocks: 64|time: 0.01304 ms|
|grids: 1|blocks: 128|time: 0.010243 ms|
|grids: 1|blocks: 256|time: 0.013312 ms|
|grids: 1|blocks: 512|time: 0.024576 ms|
|grids: 1|blocks: 1024|time: 0.004096 ms|
|grids: 8|blocks: 32|time: 0.00572 ms|
|grids: 8|blocks: 64|time: 0.004096 ms|
|grids: 8|blocks: 128|time: 0.003072 ms|
|grids: 8|blocks: 256|time: 0.004096 ms|
|grids: 8|blocks: 512|time: 0.004096 ms|
|grids: 8|blocks: 1024|time: 0.003564 ms|
|grids: 64|blocks: 32|time: 0.004096 ms|
|grids: 64|blocks: 64|time: 0.003472 ms|
|grids: 64|blocks: 128|time: 0.004016 ms|
|grids: 64|blocks: 256|time: 0.003708 ms|
|grids: 64|blocks: 512|time: 0.004596 ms|
|grids: 64|blocks: 1024|time: 0.004096 ms|
|grids: 512|blocks: 32|time: 0.004096 ms|
|grids: 512|blocks: 64|time: 0.003632 ms|
|grids: 512|blocks: 128|time: 0.003904 ms|
|grids: 512|blocks: 256|time: 0.00512 ms|
|grids: 512|blocks: 512|time: 0.00512 ms|
|grids: 512|blocks: 1024|time: 0.007468 ms|
|grids: 1024|blocks: 32|time: 0.005024 ms|
|grids: 1024|blocks: 64|time: 0.00512 ms|
|grids: 1024|blocks: 128|time: 0.004096 ms|
|grids: 1024|blocks: 256|time: 0.005322 ms|
|grids: 1024|blocks: 512|time: 0.005888 ms|
```

|grids: 1024|blocks: 1024|time: 0.012438 ms|

CPU: 0.006429 ms

n = 100`000

|grids: 1|blocks: 32|time: 0.795776 ms|

|grids: 1|blocks: 64|time: 0.283648 ms|

|grids: 1|blocks: 128|time: 0.14336 ms|

|grids: 1|blocks: 256|time: 0.074592 ms|

|grids: 1|blocks: 512|time: 0.04096 ms|

|grids: 1|blocks: 1024|time: 0.024288 ms|

|grids: 8|blocks: 32|time: 0.072448 ms|

|grids: 8|blocks: 64|time: 0.040768 ms|

|grids: 8|blocks: 128|time: 0.02144 ms|

|grids: 8|blocks: 256|time: 0.012288 ms|

|grids: 8|blocks: 512|time: 0.008192 ms|

|grids: 8|blocks: 1024|time: 0.007168 ms|

|grids: 64|blocks: 32|time: 0.012288 ms|

|grids: 64|blocks: 64|time: 0.008192 ms|

|grids: 64|blocks: 128|time: 0.006176 ms|

|grids: 64|blocks: 256|time: 0.004992 ms|

|grids: 64|blocks: 512|time: 0.00528 ms|

|grids: 64|blocks: 1024|time: 0.006144 ms|

|grids: 512|blocks: 32|time: 0.007104 ms|

|grids: 512|blocks: 64|time: 0.00512 ms|

|grids: 512|blocks: 128|time: 0.004096 ms|

|grids: 512|blocks: 256|time: 0.00512 ms|

|grids: 512|blocks: 512|time: 0.006144 ms|

|grids: 512|blocks: 1024|time: 0.009216 ms|

|grids: 1024|blocks: 32|time: 0.00704 ms|

|grids: 1024|blocks: 64|time: 0.006144 ms|

|grids: 1024|blocks: 128|time: 0.006144 ms|

|grids: 1024|blocks: 256|time: 0.006144 ms|

|grids: 1024|blocks: 512|time: 0.007232 ms|

|grids: 1024|blocks: 1024|time: 0.014272 ms|

CPU: 0.261209 ms

n = 44554433

|grids: 1|blocks: 32|time: 401.552 ms|

|grids: 1|blocks: 64|time: 133.658 ms|

|grids: 1|blocks: 128|time: 82.0335 ms|

```
|grids: 1|blocks: 256|time: 42.1325 ms|
|grids: 1|blocks: 512|time: 19.66348 ms|
|grids: 1|blocks: 1024|time: 12.1219 ms|
|grids: 8|blocks: 32|time: 41.4558 ms|
|grids: 8|blocks: 64|time: 20.2135 ms|
|grids: 8|blocks: 128|time: 11.6456 ms|
|grids: 8|blocks: 256|time: 6.18217 ms|
|grids: 8|blocks: 512|time: 3.10104 ms|
|grids: 8|blocks: 1024|time: 1.63301 ms|
|grids: 64|blocks: 32|time: 5.98416 ms|
|grids: 64|blocks: 64|time: 2.9399 ms|
|grids: 64|blocks: 128|time: 1.80608 ms|
|grids: 64|blocks: 256|time: 1.4552 ms|
|grids: 64|blocks: 512|time: 1.34326 ms|
|grids: 64|blocks: 1024|time: 1.44282 ms|
|grids: 512|blocks: 32|time: 2.13562 ms|
|grids: 512|blocks: 64|time: 1.52781 ms|
|grids: 512|blocks: 128|time: 1.31488 ms|
|grids: 512|blocks: 256|time: 1.30355 ms|
|grids: 512|blocks: 512|time: 1.29434 ms|
|grids: 512|blocks: 1024|time: 1.35373 ms|
|grids: 1024|blocks: 32|time: 1.75242 ms|
|grids: 1024|blocks: 64|time: 1.53405 ms|
|grids: 1024|blocks: 128|time: 1.29641 ms|
|grids: 1024|blocks: 256|time: 1.31038 ms|
|grids: 1024|blocks: 512|time: 1.26532 ms|
|grids: 1024|blocks: 1024|time: 1.0992 ms|
```

```
CPU: 119.645594 ms
```

Как видно из замеров, на небольших данных большое количество потоков не даёт прироста в производительности, а задействование других потоков отбирает часть ресурсов, ухудшая время выполнения программы. На больших данных виден значительный прирост производительности с использованием большого количества потоков.

Также стоит отметить, что выполнение большого количества простых операций на центральном процессоре оказалось в разы хуже по сравнению с исполнением этих действий на огромном числе потоков видеокарты.

Выводы

В результате данной работы было изучено и установлено программное обеспечение для работы с программно-аппаратной архитектурой параллельных вычислений. Также было реализован реверс вектора с использованием технологии CUDA.